

Pokémon Chronicles: Pocket Beasts

Documentación del Proyecto



Autor: Marciuc Dragos Stefan

Índice

1. Descripción del Proyecto	4
1.1. Concepto y Jugabilidad	4
1.2. Características principales	4
1.3. Entorno y Diseño	4
1.4. Mecánicas de Juego	4
1.5. Sistema de Progresión	4
1.6. Detalles Técnicos	5
2. Justificación del Proyecto	5
2.1. Interés Tecnológico	5
2.2. Selección del Género	5
2.3. Valor Formativo	5
3. Alcance del Proyecto	6
4. Alternativas Existentes	6
4.1. Pokémon Oficiales (Nintendo)	6
4.2. Pokémon Showdown	6
4.3. Fangames Varios	6
5. Stack Tecnológico	7
6. Objetivos	7
6.1. Objetivos Principales	7
6.2. Objetivos Secundarios	7
7. Desarrollo del Videojuego	8
7.1. Forma y Mecánica	8
7.2. Mecánicas Principales	8
7.3. Target	8
8. Requisitos del Sistema	9
8.1. Requisitos Funcionales	9
8.2. Requisitos No Funcionales	9
8.3. Requisitos de Interfaz	10
9. Casos de Uso	10
9.1. CU-01: Iniciar Combate	10
9.2. CU-02: Capturar Criatura	10
9.3. CU-03: Gestionar Equipo	11
10. Modelo y Diseño de la Base de Datos	11
10.1. Esquema SQLite	11
10.2. Modelo de Datos en C#	12
10.3. Relaciones Principales	13
10.3.1. Persistencia remota (Firestore)	13

11. Guía de Estilos	14
11.1. Logo	15
11.2. Paleta de Colores	15
11.3. Tipografías	15
11.4. Componentes UI	16
12. Mockups de Interfaz	17
12.1. Pantalla de Login	17
12.2. Pantalla de Login del Usuario	18
12.3. Pantalla de configuración del Usuario	19
12.4. Pantalla de login del admin	20
12.5. Pantalla del panel del admin	21
12.6. Pantalla del administrar usuarios	22
12.7. Pantalla de los datos de la partida	22
12.8. Pantalla de configuración del Admin	23
12.9. Pantalla de Flujos	24
12.10 Pantalla Principal del Usuario	24
12.11 Pantalla del Administrador	24
13. Flujo de Navegación	25
13.1. Mapa de Navegación	25
13.2. Descripción del Flujo	25
14. Manual de Usuario	25
14.1. Pantalla de Inicio de Sesión	25
14.1.1. Mensajes de Error	26
14.2. Pantalla de Registro de Nuevo Usuario	26
14.2.1. Validaciones de Registro	27
14.3. Acceso al Mundo del Juego	28
15. Pruebas y Validación del Sistema	28
15.1. Pantallas Implementadas y Estado	28
15.2. Casos de Prueba Ejecutados	29
16. Despliegue y Configuración del Backend	29
16.1. Entorno Local de Desarrollo	29
16.2. Ejecución del Servidor	29
17. Evaluación del Resultado Final	30
17.1. Objetivos Cumplidos	30
17.2. Resultados Técnicos Alcanzados	30
18. Posibles Mejoras y Extensiones Futuras	30
18.1. Mejoras Inmediatas (Prioridad Alta)	30
18.2. Ampliaciones de Contenido (Prioridad Media)	31
19. Conclusiones	31
19.1. Logros Académicos y Técnicos	31
19.2. Reflexión Final	31

20. Entregas y Observaciones Finales	31
20.1. Resumen de Entregas Realizadas	31
20.2. Adaptaciones Realizadas Durante el Desarrollo	31
20.3. Repositorio del Proyecto	32

1 Descripción del Proyecto

1.1 Concepto y Jugabilidad

“Pokémon Chronicles: Pocket Beasts” es un videojuego desarrollado en Unity que recrea las mecánicas clásicas de la saga Pokémon. El jugador podrá explorar un mundo, capturar criaturas y participar en combates por turnos, implementando un RPG funcional con sistemas completos de progresión, combate y gestión de datos.

1.2 Características principales

- **Exploración del mundo:** Movimiento libre por diferentes zonas con encuentros aleatorios
- **Sistema de combate:** Combates por turnos con IA básica enemiga
- **Captura de criaturas:** Sistema de captura con diferentes probabilidades de éxito
- **Progresión:** Sistema de experiencia, niveles y evolución de criaturas
- **Base de datos:** Gestión de datos de criaturas, movimientos y tipos
- **Persistencia:** Guardado y carga de partidas mediante SQLite

1.3 Entorno y Diseño

El juego presenta las siguientes características:

- **Estilo visual:** Gráficos 2D con perspectiva cenital en estilo pixel art retro
- **Interfaz:** HUD de combate e inventario intuitiva y clara
- **Sonido:** Efectos de audio para acciones principales y música ambiental

1.4 Mecánicas de Juego

El sistema de juego está construido sobre Unity, ofreciendo:

Acción	Implementación
Movimiento	Teclas WASD o flechas direccionales
Interacción	Tecla X y Z
Menú	Tecla ESC o botón específico
Combate	Sistema por turnos

Cuadro 1: Controles básicos del juego

1.5 Sistema de Progresión

El juego incluye un sistema de progresión basado en:

- **Experiencia:** Ganada tras ganar combates
- **Niveles:** Subida de nivel con mejora de estadísticas
- **Evoluciones:** Transformación de criaturas al alcanzar niveles específicos

1.6 Detalles Técnicos

El proyecto utiliza:

- **Motor:** Unity 6000.0.40f1 LTS
- **Lenguaje:** C#
- **Base de datos:** SQLite para persistencia local (caché/offline) y Firestore (Firebase) para sincronización remota y autenticación
- **Autenticación y servicios en la nube:** Firebase Authentication y Firestore (Google Firebase)
- **Control de versiones:** Git + GitHub

2 Justificación del Proyecto

2.1 Interés Tecnológico

El desarrollo de videojuegos es un campo en constante evolución que combina programación, diseño y creatividad. Este proyecto permite aplicar conocimientos de desarrollo de software en un contexto práctico y atractivo.

2.2 Selección del Género

¿Y por qué un RPG al estilo Pokémon?

- **Compatibilidad técnica:** Género bien documentado con mecánicas establecidas
- **Curva de aprendizaje:** Permite implementar sistemas progresivamente
- **Valor educativo:** Abarca múltiples áreas del desarrollo de software

2.3 Valor Formativo

Este proyecto ofrece:

- Desarrollo de habilidades en C# y Unity
- Implementación de arquitecturas de software
- Gestión de bases de datos y persistencia
- Control de versiones y metodologías ágiles

3 Alcance del Proyecto

El juego incluirá:

- Un mundo explorable con diferentes zonas
- Sistema de combate por turnos funcional
- En un principio 8-10 criaturas únicas con sus evoluciones
- Sistema de captura y gestión de equipo
- Persistencia de datos con SQLite
- Interfaz de usuario completa

Nota: Los elementos finales podrán ajustarse durante el desarrollo según pruebas técnicas.

4 Alternativas Existentes

4.1 Pokémon Oficiales (Nintendo)

- **Ventajas:** Juegos completos y pulidos
- **Desventajas:** Código cerrado, no permiten modificación
- **Diferenciación:** Mi proyecto es educativo y de código abierto

4.2 Pokémon Showdown

- **Ventajas:** Sistema de combate muy desarrollado
- **Desventajas:** Solo combates, sin exploración
- **Diferenciación:** Se ofrece experiencia RPG completa

4.3 Fangames Varios

- **Ventajas:** Gran variedad de ideas y enfoques
- **Desventajas:** Calidad variable, pocos documentados
- **Diferenciación:** Enfoque en arquitectura técnica y documentación

Tecnología	Uso	Versión / nota
Unity	Motor principal del juego	6000.0.40f1 LTS
C#	Lenguaje de programación	.NET 4.x
SQLite	Base de datos relacional	3.x
Firebase Authentication	Autenticación de usuarios	Firebase (SDK Unity)
Firestore	Almacenamiento y sincronización remota	Firestore (document DB)
Firebase Emulator Suite	Pruebas locales (opcional)	Recomendado en desarrollo
Git + GitHub	Control de versiones	Latest
Aseprite o variantes	Creación de sprites	Latest

Cuadro 2: Stack tecnológico del proyecto

5 Stack Tecnológico

6 Objetivos

6.1 Objetivos Principales

Los objetivos fundamentales de este proyecto son:

- **Desarrollo del juego funcional:** Crear un RPG que permita:
 - Exploración del mundo con movimiento libre
 - Sistema de combate por turnos funcional
 - Captura y gestión de criaturas
- **Sistema de base de datos:** Implementar persistencia que incluya:
 - Datos de criaturas y movimientos
 - Guardado de partidas de usuario
 - Gestión de progreso del jugador
- **Interfaz de usuario:** Desarrollar interfaces que ofrezcan:
 - HUD de combate clara e intuitiva
 - Menús de gestión de equipo
 - Sistema de navegación fluido

6.2 Objetivos Secundarios

Como metas complementarias se plantean:

- Implementar sistema de evoluciones
- Crear IA básica para combates
- Desarrollar efectos visuales y sonoros

Nota: Estos objetivos podrán ajustarse durante el desarrollo en función de los resultados obtenidos en las primeras pruebas técnicas.

7 Desarrollo del Videojuego

7.1 Forma y Mecánica

- **Género:** RPG por turnos
- **Perspectiva:** 2D cenital
- **Estilo visual:** Pixel art retro
- **Estructura:** Mundo abierto con progresión guiada

7.2 Mecánicas Principales

- **Exploración:**
 - Movimiento libre por el mundo
 - Encuentros aleatorios en hierba
 - Interacción con NPCs y objetos
- **Combate:**
 - Sistema por turnos basado en stats
 - Tipos elementales con ventajas/desventajas
 - 4 movimientos por criatura
- **Progresión:**
 - Sistema de experiencia y niveles
 - Evoluciones en niveles específicos
 - Mejora de stats por nivel

7.3 Target

- Jugadores de 12-35 años
- Fanáticos de RPGs y juegos retro
- Sesiones de 15-45 minutos

8 Requisitos del Sistema

8.1 Requisitos Funcionales

- **RF-01:** Sistema de exploración del mundo
 - Movimiento del jugador por el mapa
 - Encuentros aleatorios con criaturas
 - Transición entre zonas
- **RF-02:** Sistema de combate por turnos
 - Selección de movimientos
 - Cálculo de daño y efectos
 - IA básica para oponentes
- **RF-03:** Captura y gestión de criaturas
 - Sistema de captura con probabilidades
 - Gestión de equipo (6 criaturas máximo)
 - Almacenamiento en PC
- **RF-04:** Progresión y evoluciones
 - Sistema de experiencia y niveles
 - Evolución en niveles específicos
 - Mejora de estadísticas
- **RF-05:** Persistencia de datos
 - Guardado de partidas en SQLite
 - Carga de partidas guardadas
 - Gestión de múltiples usuarios

8.2 Requisitos No Funcionales

- **RNF-01:** Rendimiento
 - 30 FPS en hardware estándar
 - Tiempos de carga menores a 3 segundos
- **RNF-02:** Usabilidad
 - Interfaz intuitiva y clara
 - Curva de aprendizaje suave
 - Feedback visual inmediato
- **RNF-03:** Compatibilidad
 - Windows 10/11
 - Resoluciones 1080p y 720p

8.3 Requisitos de Interfaz

- **RI-01:** HUD de combate
 - Información clara de HP y estado
 - Selección intuitiva de movimientos
- **RI-02:** Menús del juego
 - Navegación con teclado
 - Diseño consistente en todas las pantallas
 - Acceso rápido a funciones principales

9 Casos de Uso

9.1 CU-01: Iniciar Combate

Actor Principal: Jugador

Precondiciones:

- Jugador explorando el mundo
- En zona de hierba o encuentro forzado

Flujo Principal:

1. Sistema detecta encuentro aleatorio
2. Transición a escena de combate
3. Mostrar HUD de combate con opciones
4. Iniciar sistema de turnos
5. Jugador selecciona movimiento
6. Resolver turno y mostrar resultados

9.2 CU-02: Capturar Criatura

Actor Principal: Jugador

Precondiciones:

- En combate con criatura salvaje
- Criatura con vida reducida
- Tener Pokéballs en inventario

Flujo Principal:

1. Jugador selecciona “Capturar”

2. Sistema calcula probabilidad de éxito
3. Mostrar animación de lanzamiento
4. Determinar resultado de captura
5. Actualizar equipo o inventario

9.3 CU-03: Gestionar Equipo

Actor Principal: Jugador

Precondiciones:

- En menú principal o centro Pokémon
- Tener criaturas en equipo o PC

Flujo Principal:

1. Acceder a menú de equipo
2. Ver estadísticas de criaturas
3. Reorganizar orden del equipo
4. Intercambiar entre equipo y PC
5. Guardar cambios automáticamente

10 Modelo y Diseño de la Base de Datos

10.1 Esquema SQLite

```
1 -- Tabla de usuarios
2 CREATE TABLE Users (
3     UserID INTEGER PRIMARY KEY AUTOINCREMENT,
4     Username VARCHAR(50) UNIQUE NOT NULL,
5     PasswordHash VARCHAR(255) NOT NULL,
6     CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP
7 );
8
9 -- Tabla de criaturas base
10 CREATE TABLE Creatures (
11     CreatureID INTEGER PRIMARY KEY,
12     Name VARCHAR(50) NOT NULL,
13     Type1 VARCHAR(20) NOT NULL,
14     Type2 VARCHAR(20),
15     BaseHP INTEGER NOT NULL,
16     BaseAttack INTEGER NOT NULL,
17     BaseDefense INTEGER NOT NULL,
18     BaseSpeed INTEGER NOT NULL,
19     EvolutionLevel INTEGER,
20     EvolvesTo INTEGER
21 );
```

```

22
23 -- Tabla de movimientos
24 CREATE TABLE Moves (
25     MoveID INTEGER PRIMARY KEY,
26     Name VARCHAR(50) NOT NULL,
27     Type VARCHAR(20) NOT NULL,
28     Power INTEGER,
29     Accuracy INTEGER,
30     PP INTEGER NOT NULL
31 );
32
33 -- Tabla de partidas guardadas
34 CREATE TABLE SaveGames (
35     SaveID INTEGER PRIMARY KEY AUTOINCREMENT,
36     UserID INTEGER,
37     PlayerData TEXT NOT NULL,
38     CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
39     FOREIGN KEY (UserID) REFERENCES Users(UserID)
40 );

```

Listing 1: Esquema de la base de datos SQLite

10.2 Modelo de Datos en C#

```

1 [System.Serializable]
2 public class Creature
3 {
4     public int CreatureID;
5     public string Name;
6     public int Level;
7     public int CurrentHP;
8     public int MaxHP;
9     public int Attack;
10    public int Defense;
11    public int Speed;
12    public int Experience;
13    public List<Move> Moves;
14    public CreatureType Type1;
15    public CreatureType Type2;
16 }
17
18 [System.Serializable]
19 public class PlayerData
20 {
21     public string PlayerName;
22     public Vector2 Position;
23     public List<Creature> Team;
24     public List<Item> Inventory;
25     public int Money;
26 }
27
28 public enum CreatureType
29 {
30     Normal, Fire, Water, Grass, Electric,
31     Fighting, Poison, Ground, Flying
32 }

```

10.3 Relaciones Principales

- `SaveGames.UserID` $\rightarrow$ `Users.UserID`
- `Creatures.EvolvesTo` $\rightarrow$ `Creatures.CreatureID`
- `PlayerData.Team` $\rightarrow$ `Creatures.CreatureID`

10.3.1 Persistencia remota (Firestore)

Además del esquema SQLite local mostrado arriba, en el despliegue actual se sincronizan algunos datos clave con Firestore para permitir copia de seguridad y acceso desde distintos dispositivos. Un mapeo simple de colecciones es:

- `users/{uid}`: perfil del usuario (username, email, role, createdAt)
- `users/{uid}/savegames/{saveId}`: partidas del usuario (JSON serializado) //No implementado aún
- `creatures/{id}`: definición canónica de criaturas (se mantiene lectura en cliente)

SQLite sigue usándose como almacenamiento local/offline y para operaciones que requieren lecturas/escrituras rápidas en local.

11 Guía de Estilos

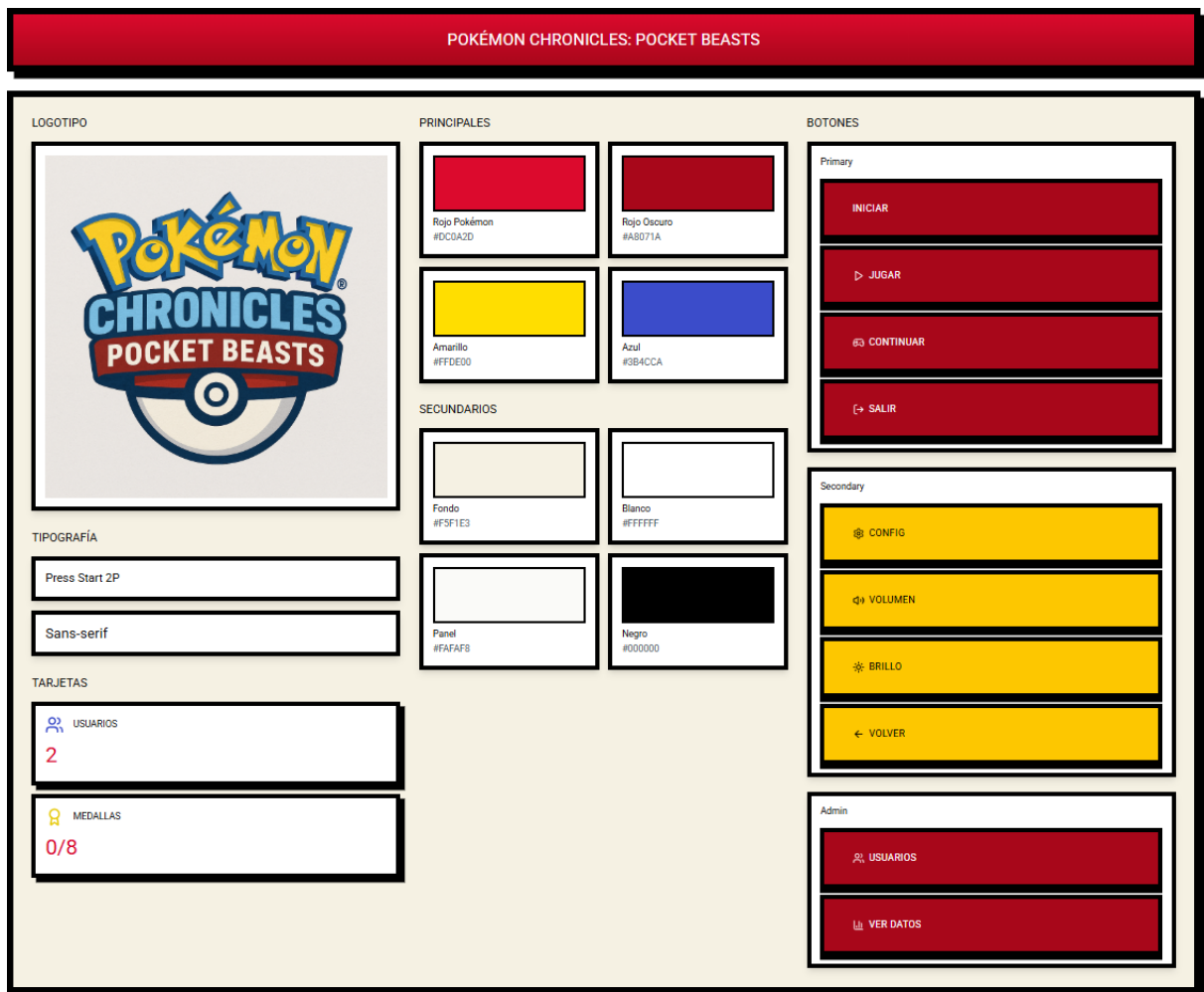


Figura 1: Guía de estilos visual completa del proyecto (colores, tipografías, componentes UI y referencias de diseño)

[Ver guía interactiva en Figma](#)

11.1 Logo

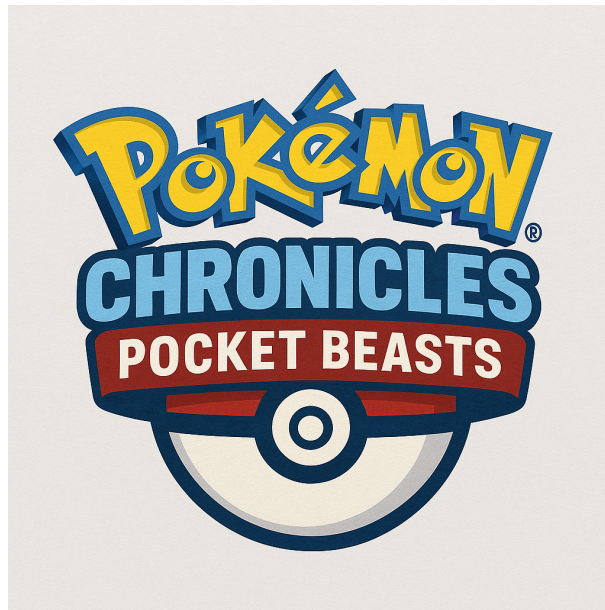


Figura 1.1: Logo principal del juego Pokémon Chronicles: Pocket Beasts

11.2 Paleta de Colores

Colores Principales

- **Rojo Pokémon (#DC0A2D)**: Color primario de los botones de acción principales como “Iniciar”, “Jugar” o “Salir”. Representa energía, pasión y el espíritu competitivo de Pokémon
- **Rojo oscuro (#A8071A)**: Variante más profunda para estados de hover o sombreado, aporta contraste visual
- **Amarillo (#FFED00)**: Usado en botones secundarios y elementos de configuración. Transmite alegría, energía y dinamismo
- **Azul (#3B4CCA)**: Representa la serenidad y equilibrio, usado en botones de administración o secciones informativas

Colores Secundarios

- **Fondo (#F5F1E3)**: Color base del fondo general, proporciona contraste y claridad a los elementos de interfaz
- **Panel (#FAFAF8)**: Paneles y tarjetas de información, aporta ligereza visual
- **Blanco (#FFFFFF)**: Texto o rellenos secundarios para resaltar botones o entradas
- **Negro (#000000)**: Contornos, tipografía y sombras principales

11.3 Tipografías

- **Títulos y botones**: Press Start 2P — tipografía pixelada, retro y distintiva, que remite a los juegos clásicos de Pokémon

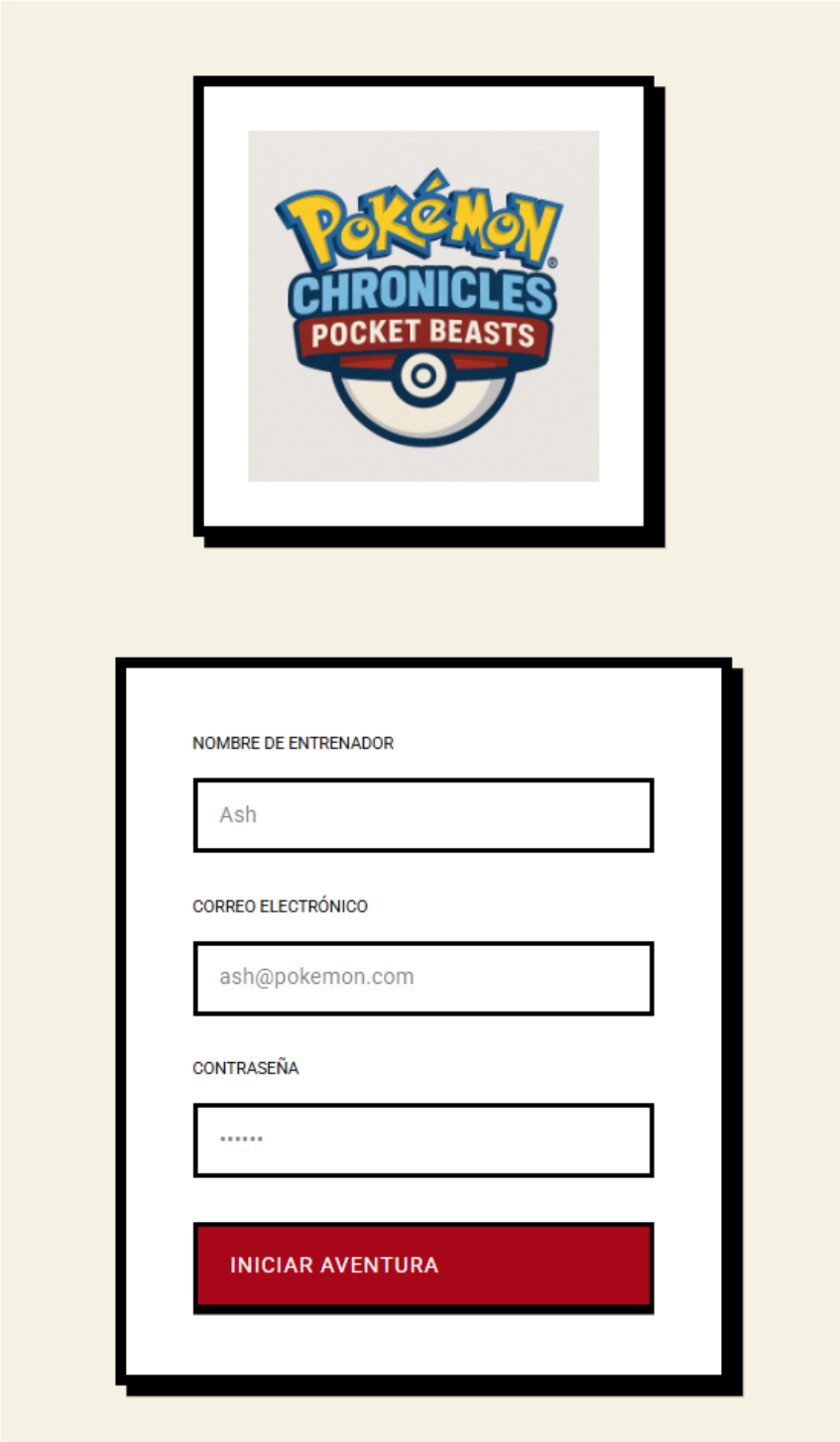
- **Texto general:** Sans-serif — utilizada para mantener la legibilidad en pantallas de diferentes tamaños

11.4 Componentes UI

- **Botones primarios:** Rojos con texto blanco, alto contraste y bordes oscuros
- **Botones secundarios:** Amarillos con texto negro, para acciones de configuración o navegación secundaria
- **Campos de texto:** Rectangulares, con bordes negros y fondo blanco para resaltar sobre el fondo beige
- **Tarjetas informativas:** Contadores de usuarios y medallas en la interfaz del administrador

12 Mockups de Interfaz

12.1 Pantalla de Login



The mockup shows a login interface for 'Pokémon Chronicles Pocket Beasts'. At the top is the game's logo, which includes the text 'Pokémon CHRONICLES' and 'POCKET BEASTS' above a Poké Ball. Below the logo is a login form with the following elements:

- Label: NOMBRE DE ENTRENADOR
- Input field: Ash
- Label: CORREO ELECTRÓNICO
- Input field: ash@pokemon.com
- Label: CONTRASEÑA
- Input field: *****
- Button: INICIAR AVENTURA

Figura 2.1: Interfaz de inicio de sesión. El jugador introduce su nombre, correo y contraseña para comenzar la aventura

12.2 Pantalla de Login del Usuario

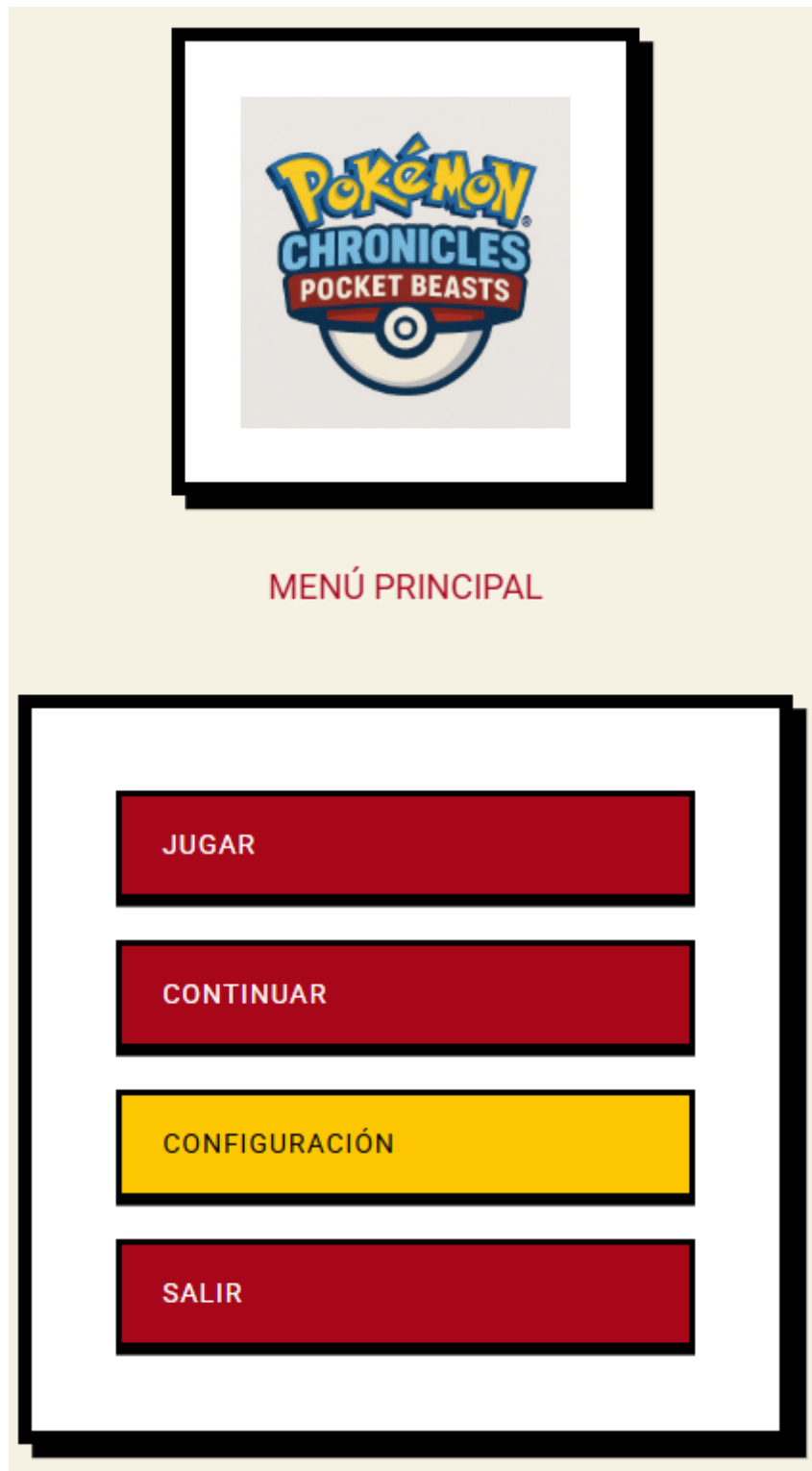


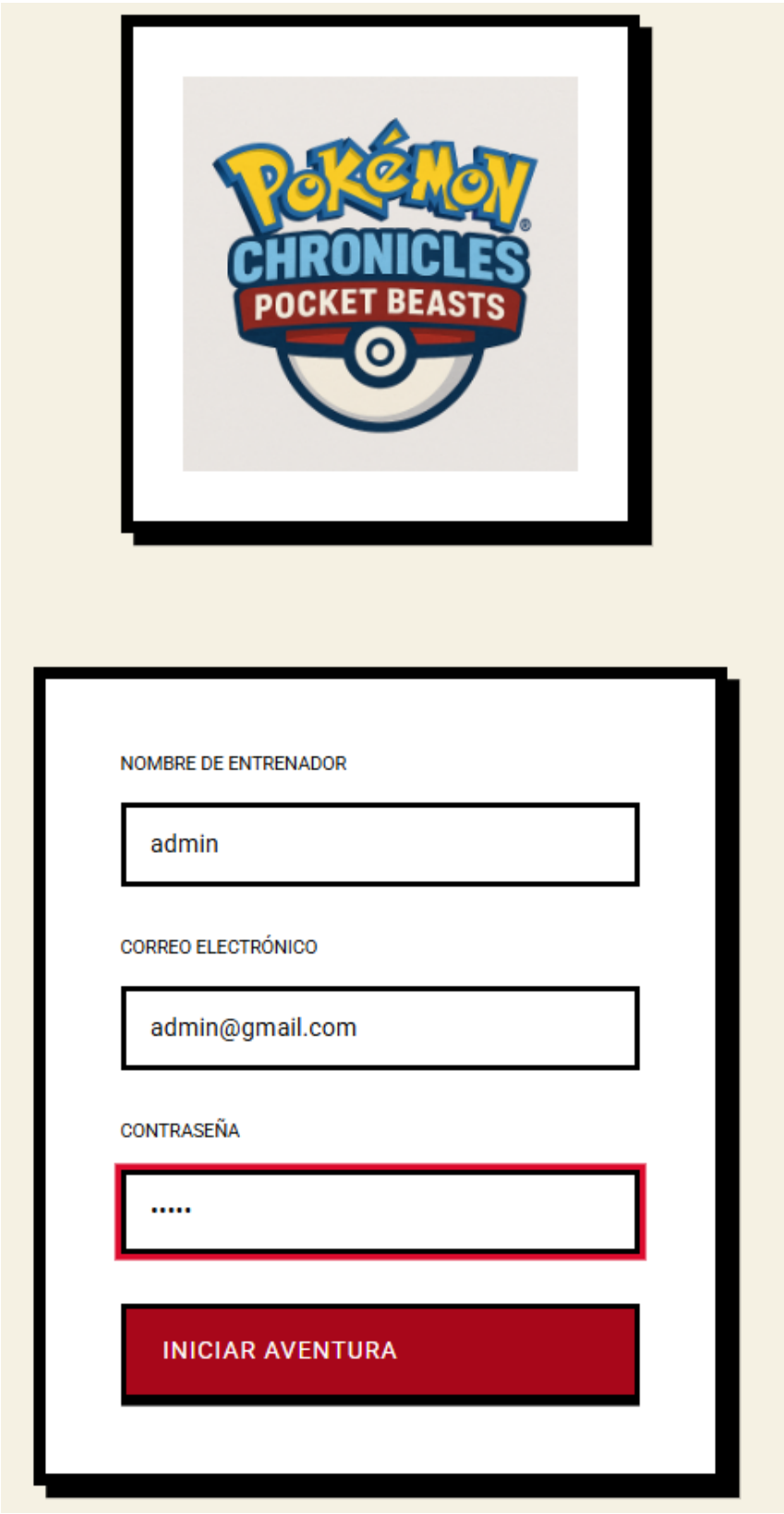
Figura 2.2: Interfaz del login del usuario normal

12.3 Pantalla de configuración del Usuario



Figura 2.3: Pantalla de configuración del usuario

12.4 Pantalla de login del admin



Pokémon
CHRONICLES
POCKET BEASTS

NOMBRE DE ENTRENADOR

admin

CORREO ELECTRÓNICO

admin@gmail.com

CONTRASEÑA

.....

INICIAR AVENTURA

Figura 2.4: Interfaz del login del admin. El nombre para entrar sería “admin” y la contraseña “admin”

12.5 Pantalla del panel del admin

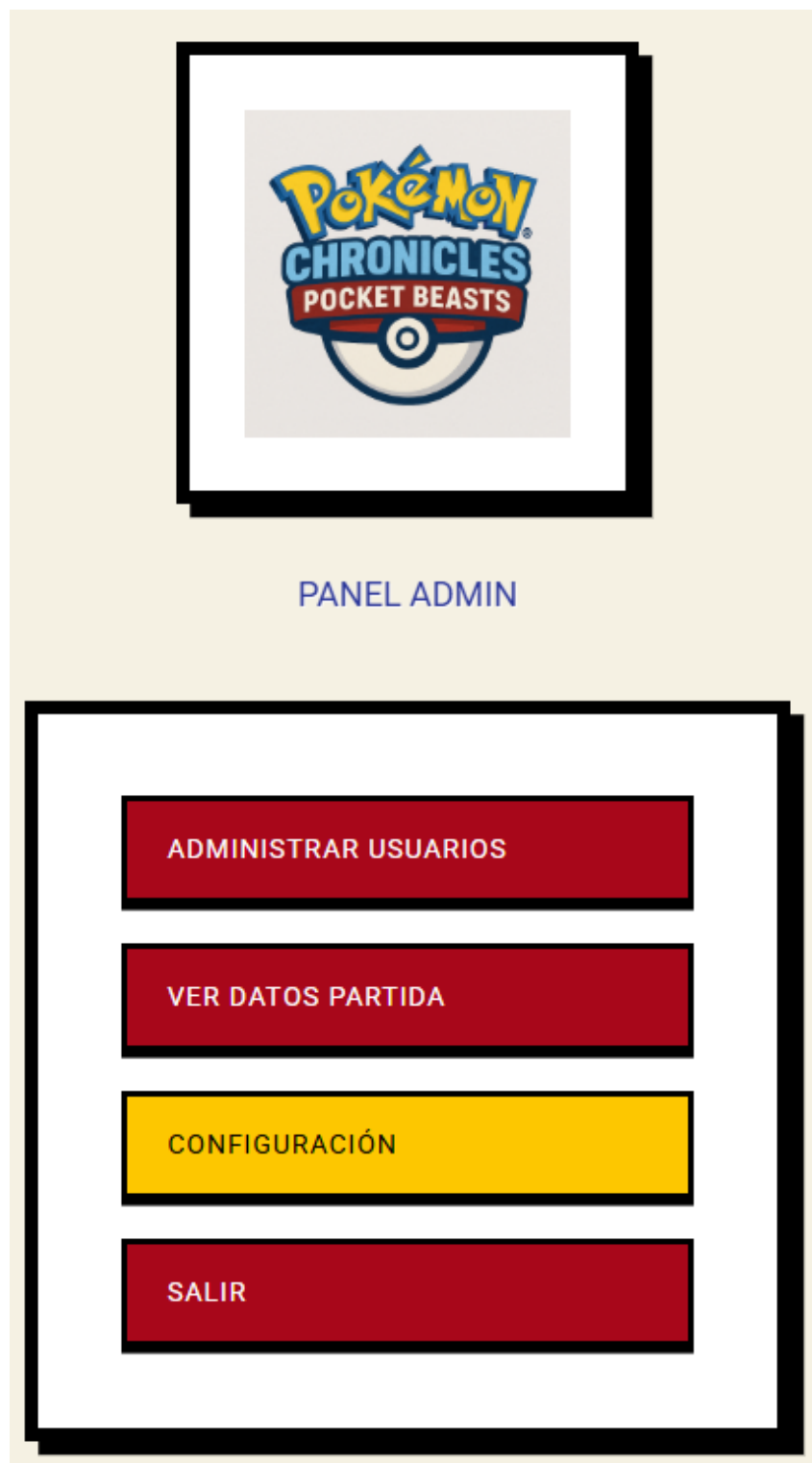


Figura 2.5: Interfaz del panel del admin

12.6 Pantalla del administrar usuarios

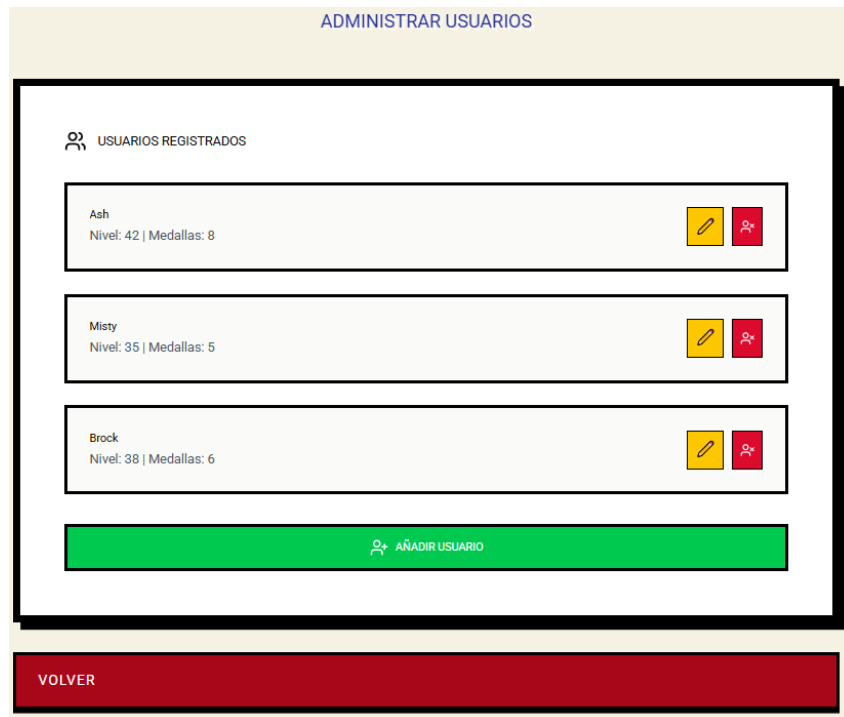


Figura 2.6: Interfaz del administrar usuarios

12.7 Pantalla de los datos de la partida

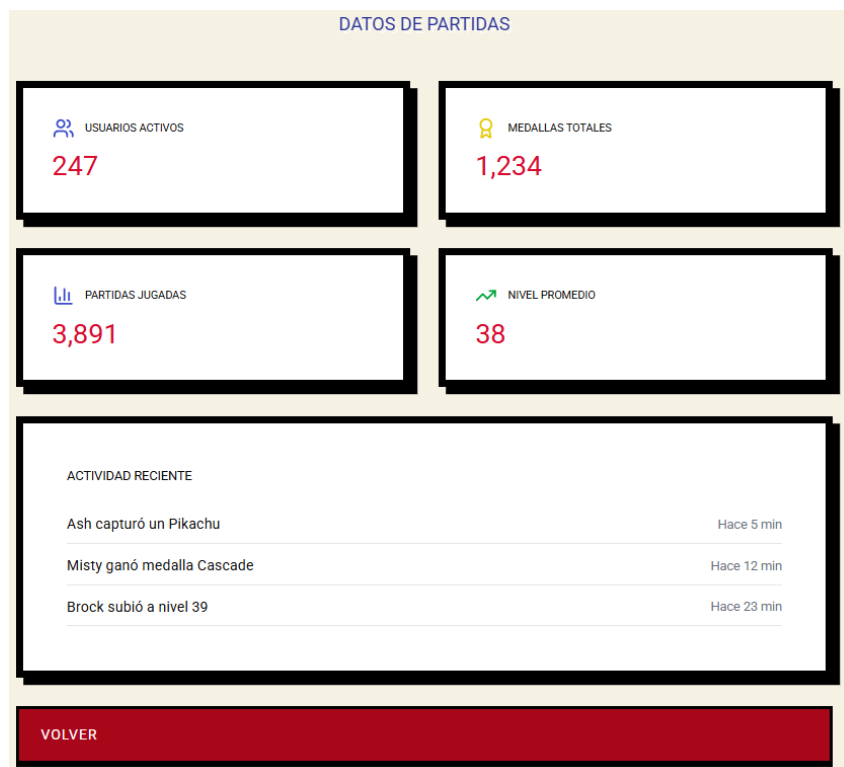


Figura 2.7: Interfaz de los datos de la partida

12.8 Pantalla de configuración del Admin



Figura 2.8: Interfaz de configuración del admin

12.9 Pantalla de Flujos

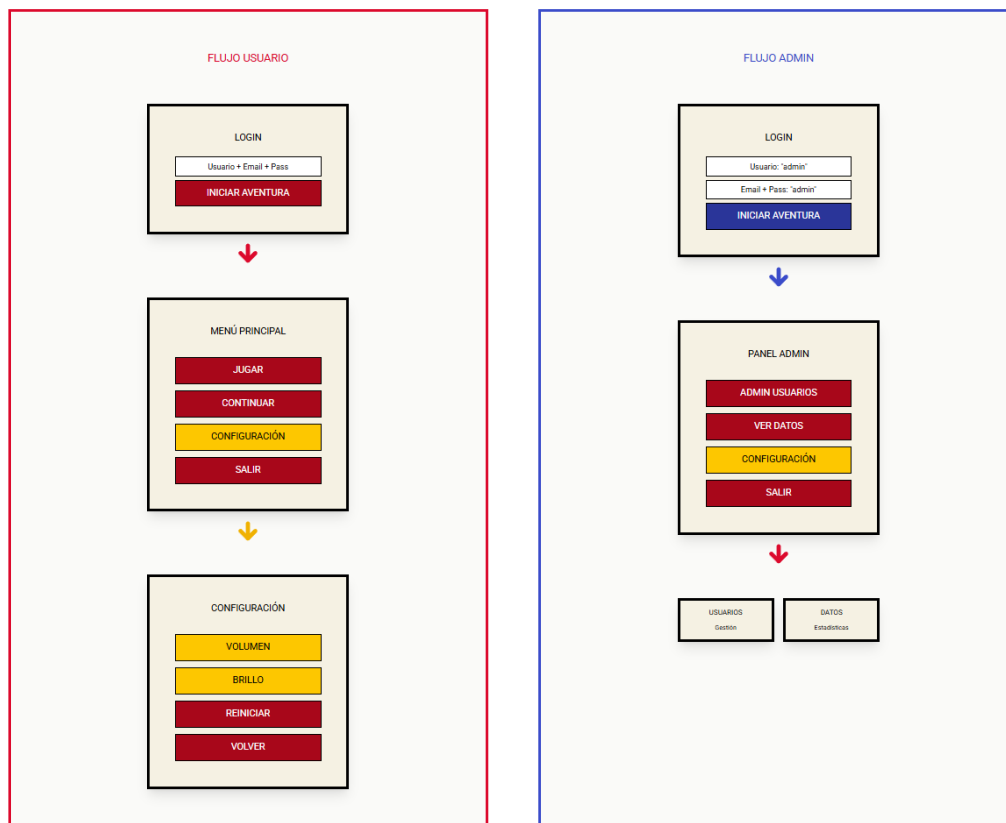


Figura 2.9: Flujos de navegación para usuario y administrador. El usuario accede al menú principal con opciones de juego y configuración, mientras que el administrador gestiona usuarios y datos

12.10 Pantalla Principal del Usuario

- **Jugar:** Inicia una nueva partida desde el principio
- **Continuar:** Retoma la partida guardada
- **Configuración:** Acceso a opciones de volumen, brillo o reinicio
- **Salir:** Cierra la aplicación

12.11 Pantalla del Administrador

- **Admin Usuarios:** Permite visualizar, editar o eliminar cuentas
- **Ver Datos:** Muestra estadísticas globales de progreso y uso
- **Configuración:** Acceso a parámetros técnicos o ajustes del sistema
- **Salir:** Finaliza la sesión administrativa

13 Flujo de Navegación

13.1 Mapa de Navegación

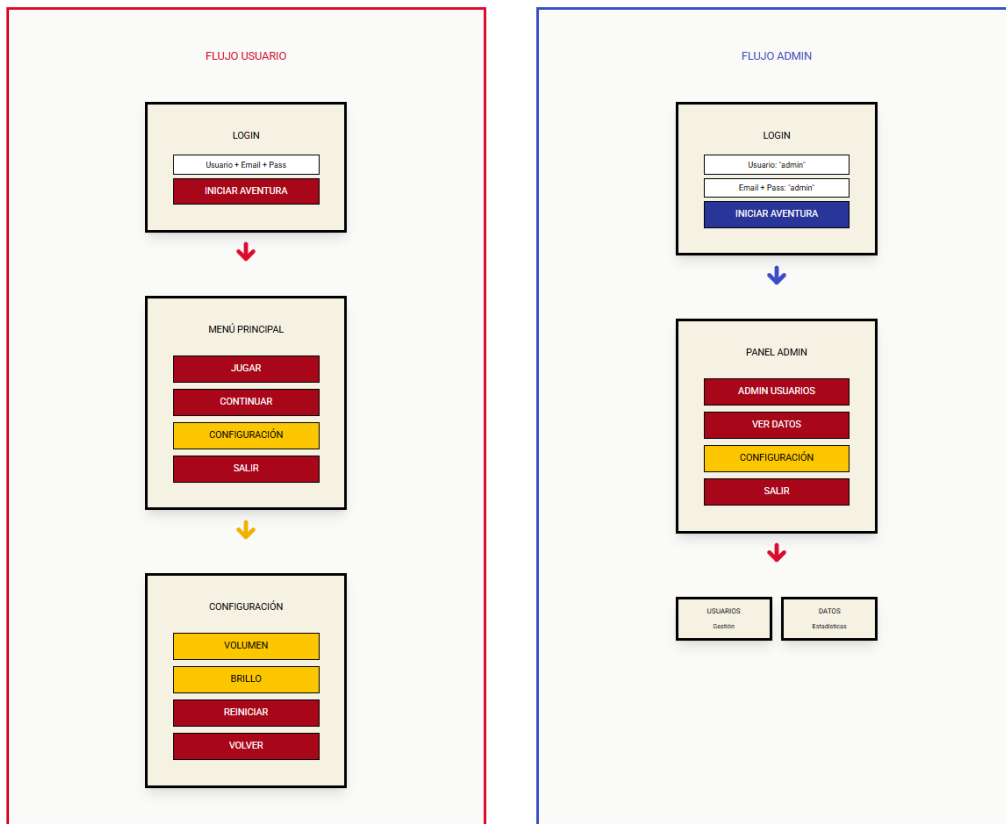


Figura 3.1: Mapa de navegación entre pantallas principales del usuario y del administrador

13.2 Descripción del Flujo

- Inicio → Login
- Si el usuario inicia sesión con credenciales normales:
 - Accede al menú principal (Jugar, Continuar, Configuración, Salir)
- Si el usuario inicia sesión con credenciales “admin”:
 - Accede al panel de administración (Usuarios, Datos, Configuración, Salir)

14 Manual de Usuario

Este manual describe el funcionamiento actual del sistema desde la perspectiva del usuario final. Incluye las pantallas implementadas, sus funcionalidades y el flujo completo de registro e inicio de sesión.

14.1 Pantalla de Inicio de Sesión

La pantalla de inicio de sesión permite al usuario acceder al sistema introduciendo sus credenciales:

- **Nombre de entrenador:** Identificador único del jugador
- **Correo electrónico:** Dirección asociada a la cuenta
- **Contraseña:** Clave de seguridad personal

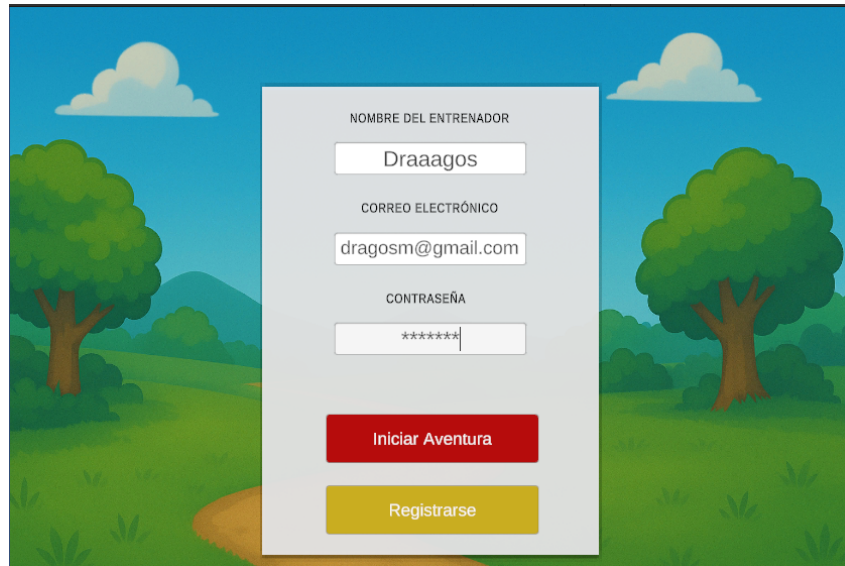
A screenshot of a login screen with a cartoonish background of green hills, trees, and a blue sky with clouds. The login form is a light gray rectangle in the center. It contains three input fields: 'NOMBRE DEL ENTRENADOR' with the value 'Draaagos', 'CORREO ELECTRÓNICO' with the value 'dragosm@gmail.com', and 'CONTRASEÑA' with masked characters '*****'. Below the fields are two buttons: a red 'Iniciar Aventura' button and a yellow 'Registrarse' button.

Figura 11.1: Pantalla de inicio de sesión con credenciales válidas

14.1.1 Mensajes de Error

Cuando las credenciales son incorrectas, el sistema muestra un mensaje de error específico:

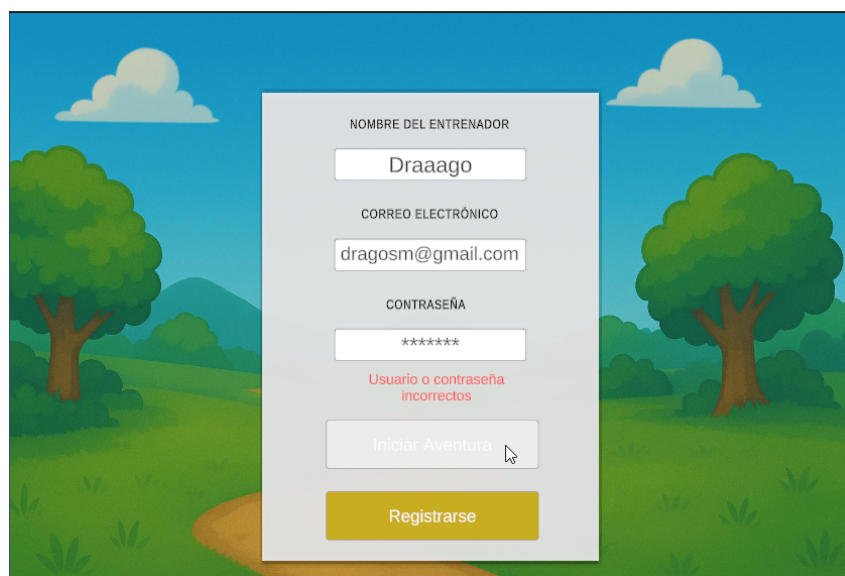
A screenshot of the same login screen as in Figure 11.1, but with an error message. The 'Iniciar Aventura' button is now disabled and grayed out. A red error message 'Usuario o contraseña incorrectos' is displayed above the button. The 'Registrarse' button remains yellow and active.

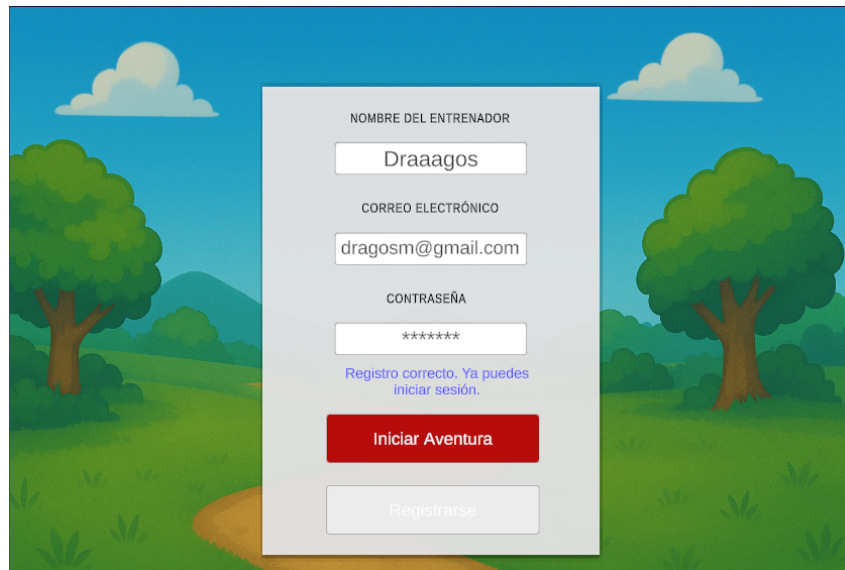
Figura 11.2: Mensaje de error tras intento de login fallido

14.2 Pantalla de Registro de Nuevo Usuario

La pantalla de registro permite crear una nueva cuenta en el sistema con las siguientes validaciones:

- **Formato de correo válido:** Debe seguir la estructura `usuario@dominio.com`

- **Contraseña segura:** Mínimo 6 caracteres
- **Usuario único:** Verificación en tiempo real con el backend



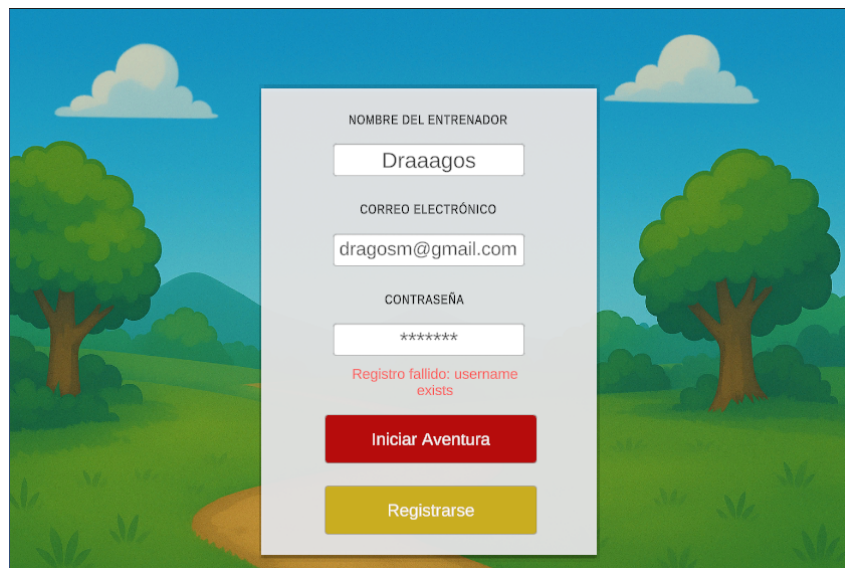
A registration form is displayed against a cartoon background of a green field, trees, and a blue sky with clouds. The form has the following fields and elements:

- NOMBRE DEL ENTRENADOR:** A text input field containing "Draaagos".
- CORREO ELECTRÓNICO:** A text input field containing "dragosm@gmail.com".
- CONTRASEÑA:** A password input field containing "*****".
- Feedback message:** "Registro correcto. Ya puedes iniciar sesión." in blue text.
- Buttons:** A red button labeled "Iniciar Aventura" and a grey button labeled "Registrarse".

Figura 11.3: Registro completado exitosamente

14.2.1 Validaciones de Registro

El sistema previene errores comunes durante el registro:



The same registration form as in Figure 11.3 is shown, but with an error message and different button states:

- NOMBRE DEL ENTRENADOR:** "Draaagos"
- CORREO ELECTRÓNICO:** "dragosm@gmail.com"
- CONTRASEÑA:** "*****"
- Error message:** "Registro fallido: username exists" in red text.
- Buttons:** The "Iniciar Aventura" button is red, and the "Registrarse" button is yellow.

Figura 11.4: Error al registrar usuario (nombre existente)

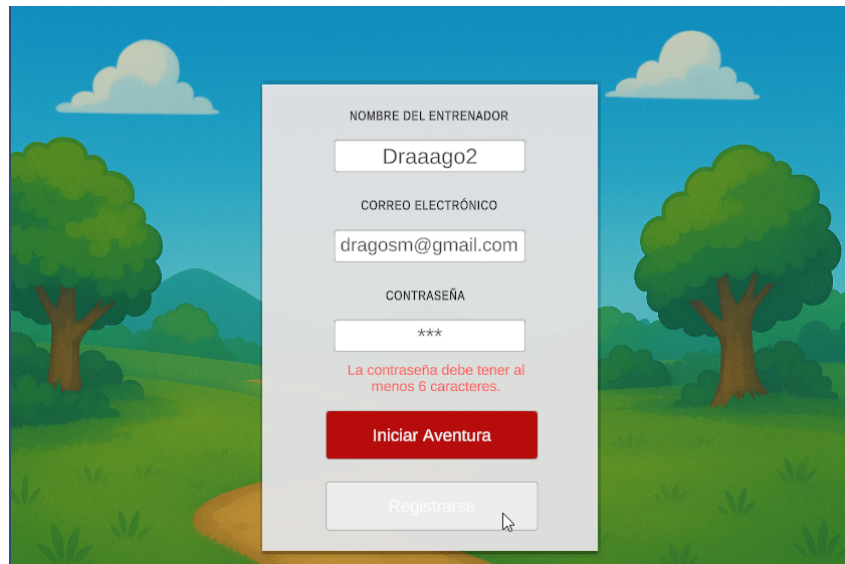


Figura 11.5: Error al registrar usuario (contraseña existente)

14.3 Acceso al Mundo del Juego

Tras un inicio de sesión exitoso, el juego carga automáticamente la escena principal del mundo:



Figura 11.6: Escena del mundo principal tras inicio de sesión exitoso

Esta pantalla incluye:

- Movimiento del personaje principal
- Elementos interactivos del entorno
- Sistema de navegación básico
- Preparación para encuentros con criaturas

15 Pruebas y Validación del Sistema

15.1 Pantallas Implementadas y Estado

La siguiente tabla resume las pantallas actualmente implementadas y su estado de prueba:

Pantalla	Funcionalidad	Estado
Inicio de Sesión	Autenticación de usuarios, validación de credenciales, comunicación con backend	Probada
Registro	Creación de nuevas cuentas, validación de datos, verificación de unicidad	Probada
Mensajes de Error	Feedback visual para errores de formato, conexión y credenciales	Probada
Escena del Juego	Mundo principal con movimiento básico y elementos interactivos	Probada

Cuadro 3: Estado actual de implementación de las pantallas del sistema

15.2 Casos de Prueba Ejecutados

- **Login exitoso:** Usuario válido → Backend devuelve token de autenticación (Firebase)
- **Login fallido:** Contraseña incorrecta → Mensaje de error visual específico
- **Registro correcto:** Datos válidos y usuario nuevo → Cuenta creada exitosamente
- **Registro fallido (usuario repetido):** Nombre existente → Mensaje “username exists”
- **Registro fallido (contraseña corta):** Validación cliente → Advertencia inmediata
- **Error de conexión:** Backend no disponible → Mensaje “Error de conexión”

16 Despliegue y Configuración del Backend

16.1 Entorno Local de Desarrollo

Durante la fase actual de desarrollo, el proyecto utiliza Firebase para autenticación y sincronización remota. SQLite se mantiene como almacenamiento local/caché. Para pruebas locales se recomienda usar Firebase Emulator Suite.

16.2 Ejecución del Servidor

En lugar de ejecutar un servidor Flask persistente, la capa de backend se apoya en servicios administrados por Firebase. Para pruebas localizadas y desarrollo se puede:

```

1 # Instalar emulador (si no esta instalado)
2 npm install -g firebase-tools
3
4 # Iniciar emulador en la carpeta del proyecto (donde esta firebase.json)
5 firebase emulators:start --only auth,firestore

```

Listing 3: Uso del emulador de Firebase

17 Evaluación del Resultado Final

17.1 Objetivos Cumplidos

El proyecto no ha alcanzado todos los objetivos principales definidos inicialmente:

- **Sistema de autenticación casi completo:** Registro, login y gestión de usuarios funcionando con Firebase
- **Exploración del mundo:** Movimiento del jugador y encuentros aleatorios implementados
- **Sistema de combate:** Mecánicas básicas de combate por turnos operativas con cálculo de daños
- **Persistencia de datos:** Aún no está implementado el guardado local mediante SQLite y sincronización remota con Firestore
- **Interfaz de usuario:** Pantallas principales diseñadas y funcionales (login, registro, combate)
- **Arquitectura cliente-servidor:** Comunicación Unity-Firebase establecida y probada
- **Roles de usuario:** Sistema de administración implementado con permisos diferenciados

17.2 Resultados Técnicos Alcanzados

Área	Resultado
Autenticación	Firebase Auth con registro/login funcional
Base de Datos	SQLite local + Firestore remoto sincronizado
Combate	Sistema por turnos con IA básica
Interfaz	3+ pantallas implementadas y navegables
Despliegue	Configuración para Firebase Cloud
Seguridad	Contraseñas hash, reglas Firestore, JWT

Cuadro 4: Resultados técnicos alcanzados

18 Posibles Mejoras y Extensiones Futuras

18.1 Mejoras Inmediatas (Prioridad Alta)

- **Sistema de evoluciones:** Implementar mecánica de evolución por nivel con animaciones
- **Ampliación de contenido:** Añadir 15+ criaturas adicionales con sus movimientos
- **IA mejorada:** Comportamiento más inteligente en combates con estrategias básicas

- **Sincronización completa:** Garantizar sincronización perfecta entre SQLite y Firestore
- **Sistema de objetos:** Implementar inventario con objetos consumibles y equipo

18.2 Ampliaciones de Contenido (Prioridad Media)

- **Más regiones:** Ampliar el mundo explorable con nuevas ciudades y rutas
- **Sistema de gimnasios:** Implementar líderes de gimnasio y medallas
- **Modo historia:** Desarrollar trama principal con NPCs y eventos
- **Sistema de crianza:** Mecánicas de reproducción y herencia de movimientos

19 Conclusiones

19.1 Logros Académicos y Técnicos

El desarrollo de **Pokémon Chronicles: Pocket Beasts** ha representado un proyecto educativo completo que ha permitido aplicar y consolidar conocimientos en múltiples áreas del desarrollo de software:

- **Programación avanzada:** Dominio de C# en Unity para desarrollo de videojuegos
- **Backend moderno:** Implementación con Firebase (Auth, Firestore, Cloud Functions)
- **Diseño de bases de datos:** Modelado relacional con SQLite y documental con Firestore
- **Desarrollo de videojuegos:** Mecánicas de juego, balance, experiencia de usuario
- **Seguridad informática:** Autenticación segura, hashing, tokens JWT
- **Gestión de proyectos:** Planificación, versionado con Git, documentación completa

19.2 Reflexión Final

Desarrollar “Pokémon Chronicles: Pocket Beasts” ha sido un reto técnico significativo que me ha permitido enfrentarme a problemas reales de desarrollo de software. A diferencia de los ejercicios académicos controlados, este proyecto me ha obligado a tomar decisiones arquitectónicas, gestionar dependencias y lidiar con la complejidad emergente de un sistema completo.

20 Entregas y Observaciones Finales

20.1 Resumen de Entregas Realizadas

20.2 Adaptaciones Realizadas Durante el Desarrollo

- **Cambio de arquitectura backend:** Migración de Flask local a Firebase Cloud para mayor escalabilidad

Entrega	Fecha	Contenido
Entrega I	24/10/2024	Documentación base, descripción, justificación, alcance
Entrega II	07/11/2024	Autenticación, prototipos Figma, guía de estilos
Entrega III	05/12/2024	Manual de usuario, pantallas implementadas, pruebas
Entrega IV	19/12/2024	Ampliación de pantallas, informes, evaluación final

Cuadro 5: Cronograma de entregas completadas

- **Ampliación de alcance:** Sistema de combate más completo del inicialmente planeado
- **Mejoras de seguridad:** Implementación de Firebase Auth en lugar de sistema propio
- **Persistencia híbrida:** Combinación SQLite local + Firestore remota para mejor experiencia

20.3 Repositorio del Proyecto

Todo el código fuente, documentación y recursos del proyecto están disponibles en el repositorio público:

<https://github.com/2DAM-dragosmarY23/Pok-mon-Chronicles-Pocket-Beasts.git>

El repositorio incluye:

- Código fuente completo del proyecto Unity
- Scripts de configuración y despliegue
- Documentación técnica y de usuario
- Assets y recursos gráficos
- Historial completo de commits y evoluciones del proyecto